A D PATEL INSTITUTE OF TECHNOLOGY

(The Charutar Vidya Mandal University)
GSIRE An ISO 9001:2015 Certified Institute

PERSONAL EXPENSE TRACKER

A seminar report submitted

In partial fulfilment of the requirements for the Degree of Bachelor of Technology

In

Information Technology

Semester – V

Subject: **Programming with Python**

By

Patel Diya Hareshbhai (12402080601039)

Faculty Guide **Nayan Mali**

Assistant Professor

Academic Year 2026-27 (Odd)

Department of Information Technology A D Patel Institute of Technology New Vallabh Vidyanagar Vitthal Udyognagar - 388121 Anand (Gujarat), India

CERTIFICATE

This is to certify that seminar work embodied in this report entitled, “PERSONAL EXPENSE TRACKER” was carried out by Patel Diya Hareshbhai (12402080601039) at A D Patel Institute of Technology for partial fulfilment of B.Tech. degree to be awarded by the Charutar Vidya Mandal (CVM) University. This seminar work has been carried out under my supervision and is to the satisfaction of department.

Date:

Place: ADIT

Guide
Nayan Mali
Assistant Professor

Head of Department
Dr. Priyang Bhatt
Professor & Head

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my seminar guide, Prof. Nayan Mali for their valuable guidance, encouragement, and support throughout the preparation of this report. I am also thankful to Dr. Priyang Bhatt, Head of Department, for providing the necessary facilities and academic environment. Finally, I extend my appreciation to my family and friends for their constant motivation.

Patel Diya Hareshbhai
(12402080601039)

INDEX

Chapter	Title	Page No.
-	Certificate	ii
-	Acknowledgement	iii
-	Abstract	2
1	Introduction	3
2	Tools And Technologies	4
3	System Design and Working	5
4	Implementation	8
5	Testing and Results	12
6	Limitations and Future Scope	13
7	Conclusion and References	14

ABSTRACT

The Personal Expense Tracker is a Python-based mini-project developed to help users record their daily spending, view it in an organised form, and understand where their money goes. The main purpose of this project is to replace manual note-keeping with a faster, simpler, and more accurate way of tracking expenses.

The program is menu-driven and offers eight options: add an expense, view all expenses, show the total, show a category summary, search, delete, view a dashboard, and exit. Each expense stores a date, description, category, and amount. The program calculates the total spending, groups spending by category, and finds the highest and average expense. It also includes input validation and exception handling, so that zero, negative, or non-numeric amounts and invalid menu choices do not crash the program.

The project uses fundamental Python concepts such as lists, dictionaries, functions, loops, if-elif-else statements, try-except blocks, user input, and string methods. It provides practical experience in modular programming and problem-solving while demonstrating how simple programming techniques can automate an everyday task. The project can be further enhanced by adding file handling, a graphical user interface, and charts.

CHAPTER 1: INTRODUCTION

The Personal Expense Tracker is a Python-based mini-project designed to automate the process of recording expenses and analysing spending. Writing expenses in a notebook and adding them up by hand is time-consuming and may lead to missed entries and arithmetic errors. This project provides a simple and consistent way to maintain a record of spending by entering the date, description, category, and amount of each expense.

The program runs as a continuous menu with eight options. It stores expenses in memory as a list of dictionaries and can display all records, the total, a category-wise summary, search results, and a small dashboard. A record can also be deleted if it was entered by mistake.

The project uses basic Python concepts such as lists, dictionaries, functions, loops, conditional statements, exception handling, and string methods. It runs through a console interface and does not require any external libraries.

1.1 Problem Statement

Manual tracking of expenses requires repeated addition and grouping of amounts. This can result in forgotten entries, calculation errors, and little awareness of spending habits. The project aims to solve this problem by recording every expense in one place and automating the calculations.

1.2 Objectives

- To accept the date, description, category, and amount of each expense.
- To validate inputs and reject zero, negative, or non-numeric amounts.
- To store all expenses as a list of dictionaries.
- To display all expenses and calculate the total spending.
- To summarise spending category-wise using a dictionary.
- To search expenses by keyword and delete a wrong record.
- To show a dashboard with the total, highest expense, and average expense.
- To use modular programming by dividing the program into functions.

CHAPTER 2: TOOLS AND TECHNOLOGIES

The project uses Python and its built-in features, making it suitable for beginners and easy to execute on different operating systems.

Component	Description
Subject	Programming with Python
Programming language	Python 3.8 or later
Code editor	VS Code, IDLE, or PyCharm
Libraries	No external libraries required
Operating system	Windows, Linux, or macOS
Interface	Console-based input and output
Error handling	try-except statements

Table 2.1: Tools and Technologies Used

2.1 Python Concepts Used

The program uses lists to store records, dictionaries to hold each expense and the category totals, functions to separate tasks, loops to repeat the menu and process records, conditional statements to make decisions, exception handling to manage invalid input, and string methods to make searching case-insensitive.

Concept	Application
Lists	Ordered collection that stores all expense records (expenses = [])
Dictionaries	Key–value pairs hold one expense or the category totals
Functions	def blocks split the app into reusable tasks
Loops	for and while repeat work over data or the menu
if-elif-else	Lets the program make decisions, such as routing the menu choice
try-except	Stops crashes caused by invalid input
User input	input() reads values; float() and int() convert them
String methods	.lower() makes searching case-insensitive

Table 2.2: Python Concepts and Their Uses

2.2 Advantages of Using Python

Python makes the project easy to develop and understand. Its built-in lists and dictionaries reduce the amount of code required, while its conditional statements and loops simplify grouping, searching, and validation. The use of functions also improves readability and makes individual sections easier to test.

CHAPTER 3: SYSTEM DESIGN AND WORKING

3.1 System Overview

The Personal Expense Tracker follows a menu-driven process in which the input is collected, validated, stored, processed, and displayed.

The system consists of four main stages:

1. Input: Collect the menu choice and, for a new expense, the date, description, category, and amount.
2. Validation: Ensure that the amount is a positive number, the delete number is in range, and the menu choice is valid.
3. Processing: Store the record, calculate totals, group by category, search, or delete.
4. Output: Display the result and return to the menu until the user chooses Exit.

3.2 System Flowchart

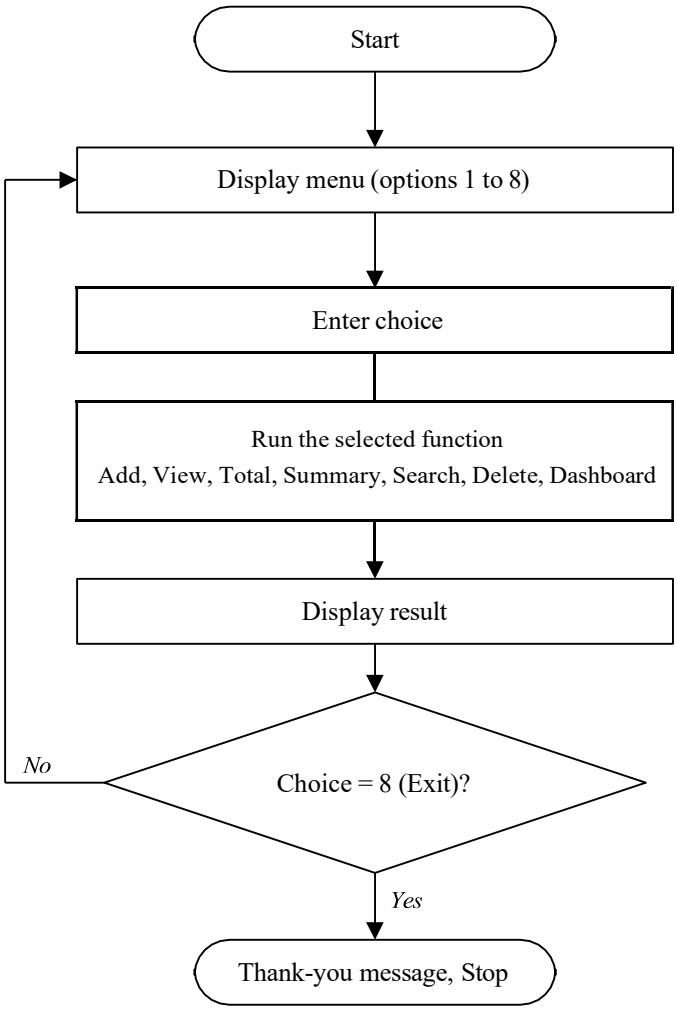


Figure 3.1: Flowchart of the Personal Expense Tracker

3.3 Menu Options

Option	Name	Purpose
1	Add Expense	Save a new record
2	View All	List every record
3	Total	Sum of all spending
4	Category Summary	Spending per category
5	Search	Find by keyword
6	Delete	Remove a record
7	Dashboard	Stats at a glance
8	Exit	Quit the program

Table 3.1: Menu Options

3.4 Data Structure

All expenses are stored in a list named expenses. Each expense is a dictionary with four keys, and a new expense is added to the end of the list using append().

```
expenses = []

expense = {
    "date":    date,
    "description": description,
    "category": category,
    "amount": amount
}

expenses.append(expense)
```

Figure 3.2: Storing an Expense

3.5 Mathematical Formulas

The program uses the following calculations to analyse spending.

Total Expense = $A_1 + A_2 + \ldots + A_n$, where A_i is the amount of the i-th expense and n is the number of expenses.

Category Total = sum of the amounts of all expenses having the same category.

Highest Expense = the largest amount among all expenses.

Average Expense = Total Expense ÷ n

Example Calculation:

Suppose the user records Food ₹200, Stationery ₹150, and Electronics ₹120.

- Total expense = 200 + 150 + 120 = ₹470
- Highest expense = ₹200 (Food)
- Average expense = 470 ÷ 3 = ₹156.67
- Category totals: Food ₹200, Stationery ₹150, Electronics ₹120

CHAPTER 4: IMPLEMENTATION

4.1 Program Structure

The program is divided into multiple functions, with each function responsible for a specific task. This makes the code easier to understand and reduces repetition.

Function	Responsibility
add_expense()	Reads the expense details, validates the amount, and stores the record
view_expenses()	Displays every record with a serial number
total_expense()	Calculates the sum of all amounts
Category summary	Groups and adds amounts by category
Search	Finds expenses matching a keyword
Delete	Removes a record by its number
Dashboard	Shows total, highest, and average expense
Main loop	Shows the menu and routes the choice to the matching function

Table 4.1: Functions Used in the Program

4.2 Adding an Expense: Input and Validation

The add_expense() function reads the date, description, and category using input(), which always returns a string. The amount is converted to a number using float(). A try-except block rejects text, and a condition rejects zero or negative amounts. In both cases the function exits early using return, so no wrong record is stored.

```
def add_expense():
    date = input("Enter date: ")
    description = input("Enter description: ")
    category = input("Enter category: ")
    try:
        amount = float(input("Enter amount: ₹"))
        if amount <= 0:
            print("Amount must be greater than 0.")
            return
    except:
        print("Please enter a valid amount.")
        return
```

Figure 4.1: add_expense() Function

4.3 Viewing and Total: Loops in Action

The view_expenses() function first checks whether the list is empty and, if so, prints a message and stops (a guard clause). Otherwise it loops through the list using for and enumerate(), numbering each expense from 1. The total_expense() function uses an accumulator variable that adds each amount to build the sum.

```
def view_expenses():
    iflen(expenses)==0:
        print("Noexpensesavailable.")
        return
    fori,expenseinenumerate(expenses,1):
        print("Expense",i)
        print("Amount      :₹",expense["amount"])

deftotal_expense():
    total=0
    forexpenseinexpenses:
        total=total+expense["amount"]
```

Figure 4.2: view_expenses() and total_expense()

4.4 Category Summary: Dictionaries

A dictionary is used as a counter. The category is the key and the running amount is the value. The in operator checks whether the category already exists; if it does, the amount is added to it, otherwise a new key is created.

```
categories = {}

for expense in expenses:
    category = expense["category"]
    amount = expense["amount"]
    if category in categories:
        categories[category] = categories[category] + amount
    else:
        categories[category] = amount

for category in categories:
    print(category, ": ₹", categories[category])
```

Figure 4.3: Category Summary Logic

4.5 Search: String Handling

The user enters a keyword, which is converted to lower case. Every expense is checked, and the keyword is compared with both the description and the category (also converted to lower case), so the search is case-insensitive and matches partial words. A flag variable named found remains False unless at least one match is printed.

```
keyword = input("Enter category or description: ").lower()
found = False

for expense in expenses:
    if (keyword in expense["description"].lower()
            or keyword in expense["category"].lower()):
        print("Description :", expense["description"])
        found = True

if found == False:
    print("No matching expense found.")
```

Figure 4.4: Search Logic

4.6 Delete and Dashboard

For deletion, the user enters the expense number. The number is rejected if it is below 1 or above the number of records. Otherwise pop(number − 1) removes the record, since list positions start from 0. The dashboard finds the highest expense by comparing each amount and calculates the average as the total divided by the number of expenses.

```
number = int(input("Enter expense number to delete: "))
if number < 1 or number > len(expenses):
    print("Invalid expense number.")
    return
deleted = expenses.pop(number - 1)

highest = expenses[0]
for expense in expenses:
    if expense["amount"] > highest["amount"]:
        highest = expense
average = total / len(expenses)
```

Figure 4.5: Delete and Dashboard Logic

4.7 Main Program: The Menu Loop

The main program uses a while True loop to keep the menu running until the user chooses option 8. An if-elif-else chain routes the choice to the matching function, and break exits the loop, which ends the program.

```
while True:
    choice = input("Enter your choice: ")

    if choice == "1":
        add_expense()
    elif choice == "2":
        view_expenses()
    # options 3-7 follow the same pattern
    elif choice == "8":
        print("Thank you for using Expense Tracker!")
        break
    else:
        print("Invalid choice! Please enter 1-8.")
```

Figure 4.6: Main Menu Loop

4.8 Input Validation and Error Handling

Input validation is an important part of the project. The program must reject invalid amounts, out-of-range delete numbers, and wrong menu choices. Instead of terminating, it displays a message and returns to the menu.

User Input	Expected Program Response
Amount: 0	"Amount must be greater than 0."
Amount: -50	"Amount must be greater than 0."
Amount: abc	"Please enter a valid amount."
Delete number: 0 or more than the record count	"Invalid expense number."
Menu choice: 9	"Invalid choice! Please enter 1-8."
View or total with no records	A message that no expenses are available
Search keyword with no match	"No matching expense found."

Table 4.2: Examples of Input Validation

4.9 Sample Output

```
========== EXPENSE TRACKER ==========
1 · Add Expense
2 · View All
3 · Total
4 · Category Summary
5 · Search
6 · Delete
7 · Dashboard
8 · Exit

Category Summary:
food        : ₹ 200.0
stationary  : ₹ 150.0
electronics : ₹ 120.0

Total Expense : ₹ 470.0
```

Figure 4.7: Sample Menu and Output

CHAPTER 5: TESTING AND RESULTS

5.1 Testing Objectives

Testing is performed to verify that the program stores and calculates expenses correctly and handles invalid input without crashing. The main objectives are to verify the total and category summary, check that search and delete work correctly, and confirm that wrong inputs only produce a message and do not stop the program.

5.2 Test Cases

Test	Input	Expected Result
Add a valid expense	Food, ₹200	Record stored in the list
View all	Three records added	All three records listed with numbers 1 to 3
Total	Food 200, Stationery 150, Electronics 120	Total Expense: ₹470.0
Category summary	Same three records	Food ₹200.0, Stationery ₹150.0, Electronics ₹120.0
Search	Keyword in description or category, in any case	Matching records displayed
Search with no match	A keyword not present	"No matching expense found."
Delete then dashboard	Three records, one of ₹120 deleted	Two expenses, total ₹350.0, highest ₹200.0, average ₹175.0
Zero or negative amount	0, -50	"Amount must be greater than 0."
Non-numeric amount	abc	"Please enter a valid amount."
Wrong menu choice	9	"Invalid choice! Please enter 1-8."
Exit	8	Thank-you message and the program stops

Table 5.1: Test Cases and Results

5.3 Analysis of Results

The test cases confirm that the program correctly stores records, calculates the total and category-wise sums, and finds the highest and average expense. The total of ₹470 matches 200 + 150 + 120, and after one ₹120 record is deleted the dashboard correctly shows ₹350 as the total and ₹175 as the average of the two remaining records. Input validation handles invalid amounts, out-of-range delete numbers, and wrong menu choices effectively.

5.4 Expected Outcomes

The expected outcome is a working console-based application that records expenses, gives the correct total and category summary, and allows searching and deleting. It should also continue running until the user chooses to exit.

CHAPTER 6: LIMITATIONS AND FUTURE SCOPE

6.1 Limitations

Although the project successfully performs basic expense tracking, it has some limitations.

- Expenses are stored only in memory, so all records are lost when the program closes.
- The date is accepted as plain text and is not checked for a valid format.
- Categories are typed freely, so “Food” and “food” are treated as different categories in the summary.
- There is no option to edit an existing record, only to delete it.
- It supports one user and has no budget limit or alerts.
- It does not include a graphical interface, charts, or a database.

These limitations do not affect the main purpose of the project but provide opportunities for future improvement.

6.2 Future Enhancements

The system can be expanded with additional features to make it more useful in daily life.

Enhancement	**Expected Benefit**
File handling (CSV or JSON)	Save expenses so they are not lost when the program closes
Date validation	Accept only valid dates using the datetime module
Fixed category list	Avoid spelling and case differences in the summary
Edit option	Correct a record without deleting it
Monthly budget	Warn the user when spending crosses a limit
Tkinter GUI	Provide a graphical interface with buttons and input fields
Charts using Matplotlib	Show spending as pie and bar charts
Database connectivity	Store and retrieve records efficiently

Table 6.1: Proposed Future Enhancements

Adding file handling would prevent results from being lost when the program closes. A graphical interface and charts could make the system easier for non-technical users, while a budget limit would help users control their spending. These improvements could transform the current beginner-level project into a more complete personal finance management system.

CHAPTER 7: CONCLUSION AND REFERENCES

7.1 Conclusion

The Personal Expense Tracker project demonstrates the practical application of Python programming in automating a common everyday task. It records expenses, calculates the total, groups spending by category, supports searching and deleting, and shows a dashboard with the highest and average expense. By reducing the need for manual calculation and note-keeping, the system helps minimise errors and encourages better awareness of spending.

The project also provides valuable experience in using lists, dictionaries, functions, loops, conditional statements, exception handling, user input, and string methods. Input validation improves reliability by ensuring that incorrect values are not stored. Testing further demonstrates that the program produces correct totals and summaries and handles invalid input without crashing.

Although the current version is limited to console-based interaction and does not store records permanently, it provides a useful foundation for future development. Features such as file handling, a graphical user interface, and charts could make the system more versatile and practical.

Overall, the project achieves its primary objective of simplifying expense tracking while strengthening the understanding of fundamental Python programming concepts.

7.2 References

1. Python Software Foundation. The Python Tutorial. Available at: https://docs.python.org/3/tutorial/
2. Python Software Foundation. Data Structures. Available at: https://docs.python.org/3/tutorial/datastructures.html
3. Python Software Foundation. More Control Flow Tools. Available at: https://docs.python.org/3/tutorial/controlflow.html
4. Python Software Foundation. Errors and Exceptions. Available at: https://docs.python.org/3/tutorial/errors.html
5. Python Software Foundation. Input and Output. Available at: https://docs.python.org/3/tutorial/inputoutput.html
6. Python Software Foundation. Built-in Types (String Methods). Available at: https://docs.python.org/3/library/stdtypes.html